Preparación de la exposición

Proyecto 02: búsqueda de claves AES-128 con Open MPI. Duración: 15 minutos. Equipo: tres personas. Material de estudio basado en el código, la serie final de 60 ejecuciones y las 13 diapositivas de Canva revisadas el 28 de octubre de 2026.

La idea que deben poder explicar

El programa prueba claves para recuperar un mensaje cifrado. La base reparte lotes consecutivos entre cuatro procesos. La alternativa cambia el orden de las candidatas, las agrupa en familias y reparte bloques mediante contadores atómicos. La base gana en la clave pequeña y en el control sin familias. La alternativa encuentra pronto las claves grandes porque pertenecen a una familia priorizada.

El cifrado usa AES-128. Las claves del experimento proceden de enteros de hasta 64 bits: no se exploró el espacio completo de AES-128.

Distribución de los 15 minutos

Persona	Diapositivas	Tiempo	Responsabilidad
1	1 a 4	0:00 a 5:00	Problema, teoría y archivos
2	5 a 8	5:00 a 10:00	Base, alternativa, avisos y método
3	9 a 13	10:00 a 15:00	Resultados, interpretación y cierre

Los nombres son intercambiables. Las tres personas deben conocer la diferencia entre speedup, búsqueda por familias y proyección. El guion marca tiempos de exposición, incluidos señalamientos de diagramas y pausas. No es un texto para leer durante quince minutos.

Cuatro cifras para recordar

Dato	Valor final
Base en 123456	0,054261 s, speedup 1,483 y eficiencia 37,1 %
Alternativa en 2^54	0,071109 s
Alternativa en 2^54 + 1	0,063716 s
Control sin familias	Speedup base 1,900 y eficiencia 47,5 %

Antes de ensayar

La versión de Canva contiene cifras y pendientes anteriores al cierre del proyecto. Las dos páginas siguientes indican qué sustituir. Este material no modifica automáticamente Canva. El informe y el ZIP de entrega mantienen su versión verificada.

Correcciones de Canva: diapositivas 1 a 8

1. Portada

Conservar el diseño del equipo. Usar el título «Búsqueda de claves AES-128 con Open MPI» para identificar el tema. Mantener los nombres actuales del grupo.

2. Objetivo y alcance

Corregir «WLS» por «WSL». Completar la frase: «es una prueba de». Texto sugerido:

- Comparar dos estrategias de búsqueda con cuatro procesos MPI.
- Mensaje: «Esta es una prueba de proyecto 2».
- Claves: 123456, 2^54 y 2^54 + 1.
- Claves AES de 16 bytes construidas desde enteros de hasta 64 bits.

3. Conceptos clave

Corregir «fUERZA BRUTA». En la validación añadir «Confirmar el acierto con HMAC-SHA256». Conservar S = T_secuencial / T_base y E = S / 4. Explicar oralmente que AES cifra y HMAC autentica.

4. Cifrado y descifrado

El esquema actual omite la autenticación. Sustituir sus pasos por: «Validar archivo y clave», «Cifrar con AES-128-ECB y PKCS#7», «Guardar AESECB2 con HMAC». En el descifrado, indicar «Verificar HMAC antes de recuperar el archivo». Si la captura dice AESECB1, usar una de las capturas finales de evidencias/inciso1/.

5. Estrategia base

Corregir «P0 validay carga» y «comparte el aciertos». Escribir «B = 65536 candidatas». La fila general de la tabla muestra inicios de lotes: llamarla «Inicio en ronda r». La barrera ocurre antes de medir, no antes de cada candidata.

6. Familias

Precisar F3: «Barrido configurable. Por defecto: inicio 2^24 y cantidad 2^63». El contador atómico impide repetir índices dentro de una familia. Las familias sí pueden coincidir en una misma clave.

7. Aviso de hallazgo

Conservar una sola diapositiva para este tema. La imagen del pseudocódigo dice «Isend»; cambiarlo por «Issend», que usa el código final. Añadir confirmación HMAC antes del aviso y drenaje de mensajes antes de liberar recursos. «No bloqueante» describe el inicio de la comunicación, no garantiza recepción instantánea.

8. Sustituir la repetición por metodología

Título: «Cómo medimos». Contenido: «5 repeticiones por combinación», «4 procesos MPI», «Corte de 5 s en búsquedas largas», «Rango base/secuencial [0, 2^56)», «Control: clave 750123, lote 4096, alternativa -e». Añadir «Medianas de tiempo de búsqueda, sin lanzamiento MPI». Hay 60 ejecuciones en total.

Correcciones de Canva: diapositivas 9 a 13

9. Resultados

Cambiar el gráfico del caso a) a estos valores, todos en milisegundos: secuencial 80,493; base 54,261; alternativa 96,404. Mantener el mismo orden y unidad en etiquetas y barras.

Caso	Secuencial	Base MPI	Alternativa
a) 123456	0,080493 s	0,054261 s	0,096404 s
b) 2^54	Sin hallazgo al corte	Sin hallazgo al corte	0,071109 s
c) 2^54 + 1	Sin hallazgo al corte	Sin hallazgo al corte	0,063716 s

Eliminar «muestras de 60 s» y los intentos de esa serie anterior. La serie final corta a 5 s. Si se muestran intentos, las medianas son: b) secuencial 9.900.032 y base 18.087.936; c) secuencial 9.519.104 y base 17.563.648.

10. Speedup y eficiencia

Sustituir los valores antiguos 1,06 y 26,4 %. La comparación medida correcta es:

Comparación	S base/secuencial	E con 4 procesos	T_base / T_alternativa
Caso a)	1,483	37,1 %	0,563
Control sin familias	1,900	47,5 %	0,736

Eliminar las filas antiguas b) y c) de speedup proyectado. Para esos casos faltan tiempos exhaustivos hasta el hallazgo. Si se conservan años estimados, son b) 288,3 secuencial y 158,5 base; c) 299,9 y 164,7. Rotularlos como extrapolaciones a tasa constante. Priorizar el control sobre esas cifras enormes.

11. Influencia de la clave

Título sugerido: «El orden de búsqueda determina cuándo aparece la clave». Corregir «Cos por candidata» por «Costo por candidata». Sustituir «inmediata» por «temprana en valores bajos». Las familias no garantizan encontrar pronto cualquier clave grande.

12. Retos resueltos y límites

Cambiar AESECB1 por «AESECB2 con HMAC-SHA256». Cambiar «-t 60» por «-t 5 en la serie final». Sustituir los pendientes por: «Clave incorrecta y alteraciones rechazadas», «Validación de rangos y consenso MPI», «Helper WSL con recuperación del estado real», «5 repeticiones y dispersión registrada». Límite vigente: WSL en una máquina, sin evaluación multinodo.

13. Conclusiones

Usar S = 1,483 y E = 37,1 % en a). La alternativa halla b) y c) en 0,071109 s y 0,063716 s. Quitar «añadir integridad» y «comparar sin familias» como pendientes: ambas tareas ya se hicieron. Recomendar ahora más claves sin estructura, otros lotes y hardware con carga controlada.

Guion de la persona 1

Diapositiva 1: presentación · 0:00 a 0:25

«Nuestro proyecto compara dos formas de repartir la búsqueda de claves AES-128 con Open MPI. Construimos una versión secuencial como referencia, una paralela por lotes y otra por familias. La pregunta es cómo influyen el reparto y el orden de candidatas en el tiempo hasta encontrar la clave».

Diapositiva 2: objetivo y alcance · 0:25 a 1:30

«Ciframos el mensaje indicado en el enunciado. La búsqueda recibe el archivo y la frase “es una prueba de”, pero no recibe la clave objetivo. Prueba candidatas hasta recuperar un texto que contiene esa frase y confirma el acierto con HMAC. Comparamos 123456, 2^54 y su vecino 2^54 + 1, con cuatro procesos MPI».

«La clave que recibe AES tiene dieciséis bytes. Nosotros generamos sus valores desde enteros de hasta sesenta y cuatro bits: los ocho bytes iniciales quedan en cero y los otros ocho contienen el número. Eso limita el espacio experimental. No estamos buscando entre todas las claves posibles de AES-128».

Señalar las tres claves. Explicar que 2^56 / 4 equivale a 2^54. La clave solo entra al programa de cifrado para preparar el caso de prueba.

Diapositiva 3: teoría · 1:30 a 3:20

«AES es un cifrador simétrico: usamos la misma clave para cifrar y descifrar. AES-128 trabaja con bloques de dieciséis bytes y una clave de dieciséis bytes. OpenSSL realiza las operaciones criptográficas. Nuestro trabajo se concentra en los archivos y en cómo organizar la búsqueda».

«El modo ECB cifra cada bloque por separado. El relleno PKCS#7 completa el último bloque y permite recuperar la longitud original. Un relleno válido por sí solo no demuestra que acertamos la clave. Por eso el archivo final incorpora una etiqueta HMAC-SHA256, que permite comprobar autenticidad e integridad».

«Cada candidata hace el mismo tipo de trabajo: construir la clave, descifrar y buscar la frase. Una clave numéricamente mayor no vuelve más difícil una operación AES. Lo que cambia es cuántas candidatas se prueban antes de llegar a ella».

«MPI usa procesos con memoria propia. Cada proceso tiene una copia del cifrado y un contexto de OpenSSL. Se comunican para recibir parámetros y detener la búsqueda. Medimos speedup como tiempo secuencial dividido por tiempo paralelo; la eficiencia divide ese resultado entre cuatro».

Diapositiva 4: archivos · 3:20 a 5:00

«aes_archivo acepta una operación, las rutas y una clave decimal. Valida entradas, conserva los bytes del texto y escribe un archivo nuevo. AESECB2 almacena versión, longitud, cifrado y etiqueta HMAC. Para descifrar, primero verifica la etiqueta. Una clave incorrecta o un archivo alterado se rechazan antes de crear una salida válida».

«La búsqueda usa la frase como filtro y confirma con HMAC únicamente cuando aparece. En cambio, el descifrado normal autentica desde el inicio. La recuperación se verificó con cmp y hashes iguales. La suite final incluye veinticuatro casos de archivos y setenta y cinco de búsqueda, además de los vectores AES».

Señalar en la captura el texto original y el recuperado. Cerrar: «Ahora explicaremos cómo distribuimos las candidatas entre los cuatro procesos».

Guion de la persona 2

Diapositiva 5: base · 5:00 a 6:25

«La base divide el rango en lotes de 65536 candidatas. En la primera ronda, P0 recorre desde cero hasta 65535; P1, desde 65536 hasta 131071; P2 y P3 continúan con los siguientes lotes. Cada proceso calcula su intervalo con el número de ronda y su identificador. Nadie tiene que asignar cada lote mediante mensajes individuales».

«P0 valida y carga el archivo, y Bcast difunde los datos. Una barrera alinea el inicio de la medición. Al terminar la ronda, Allreduce con MIN comparte la clave encontrada. Si ningún proceso acertó, acuerdan si continuar o terminar por tiempo».

«La clave 123456 está en el primer lote de P1. P1 puede encontrarla mientras los demás continúan. La base comunica el hallazgo al cerrar la ronda, así que parte del trabajo posterior no era necesario. Esa espera es una de sus limitaciones».

Señalar P1 en la tabla y el paso Allreduce. Recordar que los intervalos incluyen el inicio y excluyen el final.

Diapositiva 6: familias · 6:25 a 8:00

«La alternativa cambia el orden de exploración. F0 contiene valores bajos; F1 genera patrones como fechas, potencias y vecinos; F2 convierte cadenas cortas en números; F3 hace el barrido de respaldo. Inicialmente, cada rank prefiere una familia. Cuando la agota, toma trabajo de las siguientes en orden circular».

«P0 expone un contador por familia mediante una ventana RMA. Un proceso solicita un bloque con Fetch_and_op: incrementa atómicamente el contador y recibe su valor anterior. Si el contador era siete, ese valor identifica el bloque siete. Dos solicitudes reciben índices distintos. Win_flush completa la operación remota».

«Esto evita repetir un bloque dentro de una familia. No elimina coincidencias entre familias. Además, cada proceso mantiene su preferencia mientras tenga trabajo; no hay un planificador que elija continuamente la familia más prometedora».

«Las claves 2^54 y 2^54 + 1 aparecen en la enumeración de potencias y vecinos. Por eso una clave grande puede visitarse pronto. El algoritmo conoce la regla general para generar patrones, no la clave concreta del archivo».

Diapositiva 7: comunicación · 8:00 a 9:00

«Antes de buscar, cada proceso publica una recepción Irecv. Quien encuentra una clave confirmada inicia avisos con Issend. Los demás consultan Test cada 4096 intentos y entre bloques. Así pueden detenerse sin esperar a completar una ronda estática».

«El aviso no es instantáneo: depende del sondeo. Issend es un envío síncrono de inicio no bloqueante, y después debemos comprobar que termina. Al final acordamos la clave y el estado, drenamos avisos pendientes y liberamos la ventana y el contexto. Esto también permite manejar hallazgos simultáneos sin dejar envíos pendientes».

Diapositiva 8: método · 9:00 a 10:00

«La serie final contiene cinco repeticiones por combinación y sesenta ejecuciones en total. Las búsquedas exhaustivas grandes parten de cero, usan un rango que incluye las claves y terminan por un corte de cinco segundos. Mostramos medianas de búsqueda, sin contar el lanzamiento MPI».

«Agregamos un control con la clave 750123, el mismo rango y lote en las tres versiones, y alternativa con -e. Esa opción desactiva las familias y permite estudiar la distribución sin favorecer patrones. Mi compañero explicará qué muestran los tiempos».

Guion de la persona 3

Diapositiva 9: resultados · 10:00 a 11:20

«Para 123456, la secuencial tarda una mediana de 80,493 milisegundos; la base, 54,261; y la alternativa, 96,404. En este caso la base gana. Coordinar familias y explorar otros candidatos puede costar más que el ahorro conseguido en una búsqueda corta».

«En las claves grandes, la alternativa encuentra 2^54 en 0,071109 segundos y 2^54 + 1 en 0,063716. La base y la secuencial no las encuentran durante el corte de cinco segundos. Ese cinco no es el tiempo necesario para recuperarlas: es el tiempo que dejamos correr el experimento».

«La ventaja de la alternativa proviene de que ambas claves aparecen temprano como patrones. Son vecinas en su enumeración, por lo que la diferencia pequeña entre sus medianas no demuestra que incrementar en uno una clave mejore el rendimiento».

Señalar unidades y distinguir las celdas de hallazgo de las de corte. No comparar un corte de 5 s con 0,07 s como si ambos fueran tiempos completos de recuperación.

Diapositiva 10: métricas · 11:20 a 12:25

«En a), dividimos 0,080493 entre 0,054261 y obtenemos un speedup de 1,483. La eficiencia es 1,483 dividido entre cuatro: 37,1 %. Con cuatro procesos no obtenemos automáticamente cuatro veces la velocidad, porque hay comunicación, espera y diferencias en el trabajo hasta detenerse».

«En el control sin familias, la base tarda 0,242585 segundos frente a 0,460884 de la secuencial. Su speedup es 1,900 y la eficiencia 47,5 %. La alternativa tarda 0,329376 segundos. Con este rango y lote, el reparto dinámico tampoco supera a la base».

«Para b) y c) no tenemos speedup medido hasta el hallazgo exhaustivo. Los años estimados extrapolan una tasa corta y dependen de que se mantenga constante».

Diapositiva 11: interpretación · 12:25 a 13:10

«La posición de la clave en el recorrido explica el tiempo. Una operación AES no tarda siglos por recibir un entero grande. El recorrido ascendente tarda porque necesita visitar muchísimas candidatas. La familia de potencias cambia esa posición. Para una clave sin estructura, la alternativa podría depender del barrido y perder esa ventaja».

Diapositiva 12: retos · 13:10 a 14:05

«Durante la revisión corregimos la aceptación de claves incorrectas con relleno válido incorporando HMAC. También validamos los extremos de los rangos y coordinamos el estado final entre procesos. En WSL, el helper recupera el estado real del programa para distinguir hallazgo, rango agotado y tiempo agotado».

«Conservamos cinco muestras y dispersión estadística. El límite vigente es que medimos en WSL en una sola máquina, con recursos compartidos. No podemos generalizar estos tiempos a un clúster ni afirmar que la alternativa gana para cualquier clave».

Diapositiva 13: cierre · 14:05 a 15:00

«Completamos el cifrado y recuperación autenticada de archivos, la referencia secuencial y ambas estrategias MPI. La base fue más rápida para la clave pequeña y para el control sin familias. La alternativa recuperó pronto las claves grandes por su estructura. Aprendimos a separar el efecto de usar varios procesos del efecto de probar antes ciertas candidatas».

«Como continuación proponemos más claves sin estructura, distintos tamaños de lote y equipos con carga controlada. Eso permitiría evaluar cuándo conviene cambiar la distribución. Las búsquedas exhaustivas grandes permanecen documentadas como interrumpidas, no como hallazgos».

Teoría del cifrado, con ejemplos

AES, clave y representación

AES es simétrico: el receptor necesita la misma clave secreta para descifrar. AES-128 usa bloques de 128 bits y una clave de 128 bits. Ocho bits forman un byte, de modo que ambos tamaños equivalen a 16 bytes [1]. La biblioteca OpenSSL implementa AES; el proyecto no reimplementa sus transformaciones internas.

make_key(123456) produce estos 16 bytes en hexadecimal:

00 00 00 00 00 00 00 00 00 00 00 00 00 01 e2 40

El texto ASCII «123456» tendría otros bytes. Convertir el entero en big endian significa colocar sus bytes de mayor peso primero. Los ceros completan la longitud, pero no añaden entropía. Además, UINT64_MAX se reserva como señal de ausencia: el programa admite claves numéricas hasta UINT64_MAX - 1.

ECB y PKCS#7

ECB cifra independientemente cada bloque. Dos bloques iguales con la misma clave producen bloques cifrados iguales, por lo que puede revelar patrones [2]. El proyecto lo usa como esquema didáctico. HMAC añade autenticación, pero no elimina esa propiedad de ECB.

PKCS#7 añade de 1 a 16 bytes. Si faltan tres bytes, añade 03 03 03. Si el texto ya ocupa bloques completos, añade dieciséis bytes 10 en hexadecimal. En el archivo experimental hay 33 bytes, incluido el salto de línea final: añade quince bytes 0f y el cifrado ocupa 48 bytes. El contenedor de texto es mayor porque expresa el cifrado en hexadecimal y añade metadatos y etiqueta.

Cifrar y autenticar cumplen funciones distintas

AES transforma el texto para ocultarlo. HMAC-SHA256 permite comprobar que el contenido corresponde a la clave y no fue alterado. Un hash SHA-256 sin clave, como el usado para comparar archivos, no reemplaza HMAC.

AESECB2 contiene versión, longitud, cifrado y etiqueta. El código deriva una clave MAC a partir de la clave numérica con un dominio específico y autentica una representación canónica de versión, longitud y cifrado.

Por qué la búsqueda trata distinto el relleno

try_phrase desactiva la validación PKCS#7 al descifrar candidatas. Una clave incorrecta suele producir bytes sin sentido: se examinan los bytes de la longitud original y se busca la frase de 16 bytes. Solo si aparece se comprueba HMAC. Esto mantiene el criterio del enunciado y evita aceptar falsos positivos. decrypt_message, usado para recuperar archivos, autentica primero y luego valida el descifrado y el relleno.

MPI y la estrategia base

Proceso, rank y memoria distribuida

Un proceso tiene su propio espacio de memoria. rank es su identificador dentro del comunicador: aquí 0, 1, 2 o 3. MPI_COMM_WORLD reúne los procesos lanzados. Open MPI es una implementación de la interfaz MPI.

Aunque todos corran en una sola computadora, los procesos siguen teniendo memoria propia. Cada uno prueba claves con su propio contexto EVP y una copia del cifrado. Cuatro procesos no garantizan cuatro núcleos dedicados ni cuatro veces menos tiempo.

Qué hace cada operación

Operación	Función concreta en este proyecto
MPI_Bcast	P0 difunde validación, parámetros y archivo cifrado
MPI_Barrier	Alinea el inicio de la búsqueda antes del reloj
MPI_Allreduce + MIN	Todos reciben la menor clave encontrada
MPI_Reduce + MAX	P0 obtiene el tiempo del proceso más lento
MPI_Reduce + SUM	P0 obtiene el total de intentos de los procesos
MPI_Allreduce + MAX	Consenso de timeout o estado de salida

Reduce entrega el agregado al proceso raíz. Allreduce lo entrega a todos. Las colectivas deben ejecutarse en un orden compatible por todos los participantes. Una salida unilateral puede dejar a otros esperando.

Ejemplo manual con lotes de diez

En la ronda 0: P0 prueba [0,10), P1 [10,20), P2 [20,30) y P3 [30,40). Si la clave es 23, P2 la encuentra tras probar 20, 21, 22 y 23. Los otros procesos terminan sus lotes antes del consenso. En esta ronda realizan 10 + 10 + 4 + 10 = 34 intentos.

Sin acierto, la ronda siguiente cubre [40,80). Para un proceso p y una ronda r, el inicio es offset + (4r + p)B. El extremo derecho se recorta al final del rango. La fórmula permite repartir sin que P0 envíe cada lote individualmente.

Ejemplo real: 123456

Con B = 65536, P1 empieza en 65536. Hace 123456 - 65536 + 1 = 57921 intentos. Los otros tres completan 65536 cada uno. El total es 254529, frente a 123457 intentos secuenciales. La base puede acabar antes porque reparte el trabajo, aunque ejecute más intentos totales.

UINT64_MAX es un centinela: con MIN, cualquier clave válida es menor. Si todos aportan el centinela, no hay hallazgo. Al finalizar una ronda sin acierto, también se acuerda el timeout para que todos tomen la misma decisión.

La alternativa, paso a paso

Cuatro familias y preferencias por rank

F0 genera los enteros [0, 2^24). F1 genera patrones numéricos. F2 empaqueta cadenas de uno a cinco caracteres [a-z0-9]. F3 realiza un barrido configurable, por defecto desde 2^24 con 2^63 candidatas. No cubre automáticamente todos los enteros posibles de 64 bits.

El rank p empieza por la familia p % 4 y conserva esa preferencia hasta agotarla. Luego prueba las siguientes en orden circular. «Dinámica» describe la reserva de bloques disponibles; no implica que un algoritmo estime cuál familia tiene más probabilidad de acertar.

Reserva de bloques con RMA

RMA significa acceso remoto a memoria. MPI_Win_create expone los contadores de P0 dentro de una ventana. Los procesos acceden a ella mediante operaciones MPI, no como si todos compartieran libremente una variable de C.

Supón que el contador de F0 vale 7. MPI_Fetch_and_op suma uno y devuelve 7. Ese proceso recibe el bloque [7B, 8B). La siguiente solicitud recibe 8. La operación es atómica: dos solicitudes concurrentes no reservan el mismo índice [3]. MPI_Win_flush asegura la finalización de la operación remota.

Qué ocurre al encontrar la clave

Cada proceso deja un MPI_Irecv publicado. Un acierto confirmado inicia MPI_Issend con la clave y la familia. Los demás consultan MPI_Test cada 4096 intentos por familia y al terminar bloques. El aviso reduce el trabajo posterior, con una demora que depende del sondeo.

Issend inicia un envío síncrono sin bloquear la llamada. Su finalización exige la participación de una recepción compatible. El búfer no debe modificarse hasta que el envío termine [4]. El código conserva notice, acuerda el resultado y drena mensajes mientras MPI_Testall progresa los envíos. Luego libera ventana, solicitudes y contexto.

Ventaja y costo

Las claves grandes del enunciado aparecen en la familia de potencias y vecinos. La ganancia depende de esa elección de candidatas. Las familias se generan por índice, sin almacenar una lista enorme, pero algunas claves aparecen en más de una familia. Reservar bloques, generar patrones y sondear mensajes también cuesta tiempo.

La opción -e desactiva F0, F1 y F2. Todos reservan bloques de F3, lo que permite comparar reparto dinámico y estático sobre el mismo rango. El control conserva diferencias de asignación y parada, así que los intentos efectivos pueden variar.

Cómo interpretar los resultados

Speedup, eficiencia y rendimiento

S = T_secuencial / T_paralelo. Para a): 0,080493276 / 0,054260777 = 1,483. E = S / 4 = 0,3709, equivalente a 37,1 %. El tiempo paralelo es aproximadamente el 67,4 % del secuencial: la reducción del tiempo ronda el 32,6 %. Speedup 1,483 no significa reducir el tiempo en 148,3 %.

R = intentos / segundos. Cuenta el trabajo agregado por unidad de tiempo. R puede aumentar aunque el tiempo hasta hallar la clave empeore, si el algoritmo hace más intentos o prioriza otras candidatas.

G = T_base / T_alternativa. En a), G = 0,563: la base tarda menos. En el control G = 0,736: también gana la base. Se debe escribir la fórmula porque «G alt/base» puede resultar ambiguo.

El control

Clave 750123, rango [0,1000000), lote 4096 y alternativa -e. Las medianas son 0,460884 s secuencial, 0,242585 s base y 0,329376 s alternativa. La base obtiene S = 1,900 y E = 47,5 %. Este resultado describe esa configuración; no demuestra que el reparto estático gane siempre.

Medir cinco segundos no equivale a recuperar en cinco segundos

Para b) y c), la base y la secuencial retornan estado 3: tiempo agotado. El rango [0,2^56) sí incluye las claves. La base revisa el corte entre rondas, de modo que la duración puede superar un poco los cinco segundos.

La proyección usa T_estimado = (clave + 1) / mediana(R). Por ejemplo, para b) base: (2^54 + 1) / 3600815, aproximadamente 158,5 años. Supone rendimiento constante durante muchísimo más tiempo que la muestra. No es un hallazgo ni una predicción precisa de años de ejecución.

Qué factores impiden obtener S = 4

Comunicación y colectivas, espera al final de las rondas, trabajo extra antes de parar y recursos compartidos de WSL. Los bloques pequeños requieren más coordinación; los grandes demoran la parada. No se midió un estudio de escalabilidad con distintas cantidades de procesos o nodos.

Mediana y calidad de la comparación

Cada combinación tiene cinco muestras. La mediana es el valor central después de ordenarlas y reduce la influencia de valores extremos. La dispersión sigue siendo necesaria para observar variabilidad. No afirmar significancia estadística por diferencias pequeñas. Los relojes miden búsqueda, no el tiempo completo de lanzar y cerrar la aplicación.

Preguntas probables y respuestas breves

¿Rompieron AES-128?

No. AES usa una clave de 128 bits, pero las candidatas del proyecto proceden de enteros de hasta 64 bits. Además, dos claves de prueba tienen una estructura que la alternativa prioriza.

¿Cómo sabe la búsqueda que acertó sin conocer la clave?

Descifra con cada candidata, busca «es una prueba de» y confirma con HMAC. El programa de cifrado recibe la clave para crear el caso; los programas de búsqueda no la reciben.

¿HMAC les entrega la clave o hace trampa?

No la revela. Permite comprobar una candidata. El código mantiene la búsqueda de la frase como filtro, conforme al enunciado. La autenticación también evita aceptar una clave errónea por casualidad.

¿Por qué añadieron HMAC si ya estaba PKCS#7?

PKCS#7 permite validar el relleno, pero una clave equivocada puede producir relleno aparentemente válido. HMAC autentica el archivo. La regresión con la clave 44 comprueba el rechazo sin crear salida.

¿Por qué no usar HMAC directamente en cada candidata?

Sería otro diseño de verificación posible con este formato. Aquí conservamos el descifrado y la frase exigidos en el enunciado; HMAC confirma los posibles aciertos. No comparamos ambos diseños de rendimiento.

¿Qué evita que dos procesos tomen el mismo bloque?

En la base, la fórmula de intervalos disjuntos. En la alternativa, el incremento atómico del contador de cada familia. Esto no elimina candidatas repetidas entre familias.

¿Por qué la alternativa pierde en la clave pequeña?

La búsqueda ya es corta. El trabajo de otras familias y la coordinación pueden superar el ahorro de la parada temprana. El control sin familias tampoco mostró ventaja dinámica con ese lote y rango.

¿Por qué b) y c) tardan distinto si son vecinas?

Sus posiciones en patrones son cercanas. Las muestras dependen de planificación, comunicación y recursos compartidos. La diferencia observada no demuestra un efecto estable de sumar uno.

¿MPI_Issend termina inmediatamente?

La llamada inicia el envío sin bloquear. La operación sigue pendiente y debe completarse con una recepción compatible y comprobaciones posteriores. No bloqueante no significa entrega instantánea.

¿El proyecto sigue incompleto por los timeouts?

Las funciones y la evidencia están completas. Los recorridos exhaustivos grandes no terminaron por su costo. Se documentan como interrumpidos y se mantienen separados de las recuperaciones reales de la alternativa.

Ensayo, demostración y autoevaluación

Sesión de estudio de 45 minutos

- Minutos 0 a 10: explicar sin leer qué hace el proyecto, cómo forma la clave y por qué usa HMAC.
- Minutos 10 a 20: dibujar en papel dos rondas de la base y simular una reserva atómica de la alternativa.
- Minutos 20 a 35: ensayo completo con las diapositivas corregidas. Cambiar de expositor a los minutos 5 y 10.
- Minutos 35 a 45: responder las preguntas anteriores por turnos y repetir las respuestas que se confundan.

Demostración opcional, fuera del guion de 15 minutos

En Ubuntu o WSL, desde la raíz del repositorio. Usar los archivos cifrados incluidos evita crear entradas durante la presentación:

./proyecto02/build/busqueda_secuencial -l 1000000 evidencias/inciso2/datos/clave_a.aes

python3 scripts/mpi_utils.py --run proyecto02/build/busqueda_base -l 1000000 evidencias/inciso2/datos/clave_a.aes

python3 scripts/mpi_utils.py --run proyecto02/build/busqueda_alternativa evidencias/inciso2/datos/clave_b.aes

Mostrar «Clave encontrada», mensaje y «Estado MPI: 0». Los tiempos de la demostración no sustituyen las medianas. Para enseñar un corte, usar la base con -l 72057594037927936 -t 5 y clave_b.aes. Se espera estado 3, no un hallazgo. Si la demostración es obligatoria dentro de los 15 minutos, recortar el guion y reservarle un minuto ensayado.

Comprobación sin apuntes

1. ¿Qué diferencia hay entre el tamaño de la clave AES y la cantidad de candidatas que generamos?
2. Con B = 10 y clave 23, ¿qué proceso la encuentra y cuántos intentos totales hace la base?
3. Si un contador devuelve 7, ¿qué bloque recibió el proceso?
4. Con T_sec = 0,08 s y T_MPI = 0,05 s, ¿cuáles son S y E con cuatro procesos?
5. ¿Qué significa «sin hallazgo al corte»? ¿Permite calcular speedup de recuperación?

Respuestas: (1) 128 bits de longitud, con valores restringidos por el generador numérico. (2) P2; 34 intentos en total. (3) [7B,8B). (4) S = 1,6 y E = 40 %. (5) La ejecución acabó antes de encontrar la clave; falta un tiempo completo para ese speedup.

Fuentes para profundizar

[1] NIST, FIPS 197: https://csrc.nist.gov/pubs/fips/197/final

[2] NIST, SP 800-38A: https://csrc.nist.gov/pubs/sp/800/38/a/final

[3] Open MPI, MPI_Fetch_and_op: https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Fetch_and_op.3.html

[4] Open MPI, MPI_Issend: https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Issend.3.html

Fuentes internas: proyecto02/src/, proyecto02/include/, evidencias/inciso2/resultados.json y resultados.md. Secuencia revisada: https://www.canva.com/design/DAHZRHyFVog/5nDPKcws3ZY9al8T9zFJ0g/edit